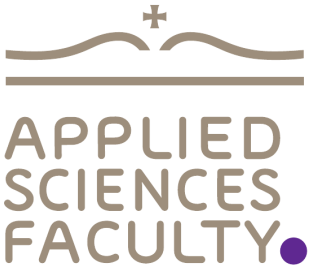
C++ — лекція 10. Структури, об'єднання, бітові поля — 1





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Структури

На противагу масивам, які є об'єднаннями елементів одного типу, структури об'єднують різнотипні елементи майже будь-яких типів:

```
struct SomeStructure
{
    int field1 ;
    double field3 ;
    int arfield [50];
    char *str ;
};
```

Крапка з комою внизу — важлива[1]!

Оголошення структури **створює новий тип**. Ми можемо оголошувати змінні цього типу:

```
SomeStructure var;
```

[1] Справа в тому, що свіжооголошений тип можна зразу використати для створення змінних. Список яких йтиме після }, перед крапкою з комою. Може не бути жодної змінної, як у прикладі вище, однак, якщо пропустити ";", то компілятор вирішить, що наступний ідентифікатор є іменем змінної, що призведе до помилки.

- Елементи структури називають **полями** або **членами даних**.
- Щоб звернутися до якогось поля **екземпляру структури** використовують оператор ".":

```
SomeStructure var;
var . field3 =5789.3;
```

- Якщо є вказівник на структуру, звернутися до її поля можна за допомогою оператора ->:

```
SomeStructure *sptr = new SomeStructure;
sptr->field3=5789.3;
```

- **Увага!** Оператор "." має вищий пріоритет, ніж оператор розіменування, "*". Тому:

```
*sptr . field3 ;
```

еквівалентне:

```
*( sptr . field3 );
```

- Розмір структур може не дорівнювати сумі розмірів елементів, через вирівнювання[2].
- ◇ При чому, можлива ситуація, коли нащадок використовує байти, додані в кінці предка для вирівнювання, для своїх членів[3].
- Однакові за будовою структури все рівно вважаються **різними** типами!

```
struct A{
    int data;
};
struct B{
    int data;
};
A a, a1;
B b, b1;
// a=b; // ЗАБОРОНЕНО!
a = a1;
b = b1;
```

[2] Трішки подробиць: https://www.shincbm.com/embedded/2022/02/18/struct-layout-code-size.html, вплив на продуктивність: https://johnnysswlab.com/software-performance-and-class-layout/.

[3] Приклад див. https://quuxplusone.github.io/blog/2023/02/24/trivial-swap-x-prize/, але це доволі складна технічна стаття.

- Ім'я структури можна використовувати в її оголошенні, наприклад, щоб зберегти вказівник на неї ж:

```
struct List{
    List* next;
    List* prev;
};
```

- Якщо декілька структур повинні посилатися одна на одну, ім'я однієї із них слід оголосити до її визначення:

```
struct A;
struct B{
    int data;
    A* ptr_A;
};
struct A{
    int other_data;
    B* ptr_B;
};
```

- Елементами структури можуть бути лише повністю визначені типи. Самого оголошення:

```
struct A;
```

  недостатньо[4] — для кожного поля потрібно знати його розмір. Як згадувалося раніше, вказівники на не повністю визначені типи припустимі – розмір вказівника не залежить від розміру і властивостей структури.

- Оголошуючи структуру, можна зразу оголошувати змінні відповідного типу:

```
struct A{
    int n;
    double wt;
} a1, a2;
```

[4] https://www.sandordargo.com/blog/2024/05/15/when-includes-are-needed – ще приклад, коли попереднього оголошення недостатньо.

- Структури можуть бути великими, **передача їх по значенню** може бути затратною — під час передачі аргументів відбуватиметься копіювання великих об'ємів даних. Щоб уникнути цього, можна передавати у функцію **вказівник** або **посилання (reference)** на структуру. Якщо в завдання функції не входить змінювати передані дані, слід оголосити такі параметри константними.

```
struct A{
    int data;
};

int some_fn_p(const A* in_a_ptr) // Передаємо вказівник
{ ... }

int some_fn_r(const A& in_a_ref) // Передаємо посилання
{ ... }
```

Структури можна ініціалізувати способом, схожим до ініціалізації масивів:

```
struct A{
    int n;
    double m;
    char st [5];
};

A a = {5, 3.13, "abc"};
```

Масиви структур

- Можна створювати масиви структур:

```
struct A{
    int n;
    double m;
};
A arr[11];
arr[2].m = 3.14;
arr[1] = arr[2];
```

- Можна комбінувати структурну ініціалізацію масивів та структур:

```
struct B{
    int x;
    char str[5];
};
B bar1[3] = { {1, "ab"}, {2, "cd"}, {3, "ef"} };
```

Внутрішні дужки необов'язкові:

```
B bar2[3] = { 1, "ab", 2, "cd", 3, "ef"};
```

- Після останнього елемента ініціалізації може бути кома – як для структур, так і для масивів. Іноді це зручно: "On the virtues of the trailing comma".

Структури в динамічній пам'яті

- Можна розміщати їх в динамічній пам'яті, в тому числі – в динамічних масивах. Приклад:

```
struct A{
    double m;
    int st[5];
};

A *p1 = new A;
p1->m = 5.14;

A *p2 = new A[10];
p2[2].m=2.34;
p2[3].st[2]=11;
```

Масиви у структурах

- Навпаки, структура може містити масиви. При цьому, масив розглядається, як єдиний об'єкт, копіюється при копіюванні структури:

```
struct A{
    double m;
    int st [5];
};
A a1, a2;
  ........
a2 = a1;
```

- Копіювання "звичайних" структур здійснюється побітово. Про копіювання класів буде окрема розмова.

Вкладені структури

- Оголошувати структури можна в тілі функції. Такі типи будуть локальні для неї.
- Структури можуть містити вкладені структури.

```
struct A{
    double m;
    int st[5];
    struct B{
        short z1;
        long z2;
    } x;
    B b;
};

   .....
A a;
A::B b2;
// B b3;  // Error!
```

Безіменні вкладені структури

- Вкладені структури можуть не мати імені:

```
struct A{
    double m;
    int st[5];
    struct {
        int anon1;
        int anon2;
    } as;
};

.....
A a;
a.as.anon1 = 5;
```

- Крім безіменних (unnamed) структур, в C11 існують анонімні (anonymous) структури. Воно підтримується деякими компіляторами, як нестандартне розширення до C++. Щоб "отримати", заберіть також ім'я as – в C11, об'єкт все рівно створюватиметься неявно.
- Не використовуйте його без крайньої потреби. Детальніше див., наприклад: `http://stackoverflow.com/questions/14248044/are-anonymous-structs-standard-and-really-what-are-they`.

Відмінності від C – 1

- В C структури (як і об'єднання та бітові поля), існують в окремому просторі імен. Тому оголошувати екземпляри структур слід так:

```
struct A{
    double m;
    int st[5];
};

.....
struct A a;
```

  Така форма припустима, але надмірна в C++.

- В C її теж недолюблюють, тому часто використовують наступний трюк:

```
typedef struct A A;
A a; // <== struct більше не потрібне.
```

Відмінності від C – 2

- В C++ вкладені структури видимі лише в контексті зовнішньої структури (приклад: `A::B x;`). В C вкладені структури видимі безпосередньо:

```
struct A{
    double m;
    int st[5];
    struct B{
        short z1;
        long z2;
    } x;
    struct B b;
};
.....
    struct A a;
    struct B b; // <======
```

Відмінності від C – анонімні структури C11

- На додачу до розглянутих вище безіменних структур, C11 підтримує анонімні структури, які часто доступні як нестандартне розширення компілятора і в C++[5].

```
struct C
{
    double c;
    struct{
        int a1, a2;
    };     // <== Зауважте відсутність імені змінної!
};

    ..............
struct C c;
c.a1 = 7;
```

- В C++ така поведінка підтримується хіба для об'єднань.

[5] Зрозуміло, що не варто без крайньої потреби користуватися нестандартними розширеннями!

Структури і класи

- В C++ ключове слово struct може бути використаним і для оголошення класів. Про цю можливість, та його особливості, говоритимемо, коли мова дійде до класів.

Union

- Об'єднання оголошуються схоже до структур:

```
union U{
    int  n;
    double m;
    char  st [5];
};

union U u1;
```

- Однак, на відміну від структур, всі їх поля розташовуються в одній і тій же пам'яті.
- Внаслідок цього, в кожен момент часу, union може зберігати дані лише одного типу.
- Звертатися до інших імен полів – невизначена поведінка.
  - ◇ Часто компілятори реалізовують нестандартне розширення, коли таке звертання виконує своєрідний reinterpret_cast – вміст пам'яті одного типу інтерпретується як змінна іншого типу.
- Розмір об'єднання встановлюється достатнім, щоб вмістити поле найбільшої довжини.
- Фактично, об'єднання можна вважати структурою, всі елементи якої мають нульове зміщення.

Взаємодія із класами – 1

- Об'єднання може мати функції-члени, включаючи конструктори та деструктори.
- Може мати статичні дані та функції.
- Нестатичні члени даних не можуть бути посиланнями.
- Не може мати базові класи чи бути базовим класом.
- До C++11 – не міг включати члени із нетривіальними спеціальними функціями – конструкторами копіювання, операторами присвоєння чи деструкторами.
- З C++11 – якщо є член із нетривіальними спеціальними функціями, включаючи операції з переміщеннями, вони тоді неявно видалені для об'єднання – але можна реалізувати самостійно.
- Те ж стосується конструктора за замовчуванням.
- Якщо член об'єднання має створені користувачем конструктори та деструктори, викликати їх доведеться вручну!
- ◇ Звичайне присвоєння скомпілюється – але буде помилкою.

Взаємодія із класами – 2

- Приклад:

```
union test_2_ut {
    std :: string  str ;
    std :: vector<int> vec;
    ~test_2_ut() {}
    // Оскільки str і vec мають нетривіальні деструктори,
    // потрібно оголосити явно ~test_2_ut()
    // і користі з нього буде не дуже багато ––
    // він не має способу визначити активний член.
};


    .....
    test_2_ut u1{std::string{"abc"}};
    // u1.vec = std::vector<int>{1, 2, 3};
    // ^ Скомпілюється, але –– помилка!
    u1.str.~basic_string(); // std::string == std::basic_string<char>
    new( &u1.vec ) std::vector<int>{ 1, 2, 3 };
```

- Детальніше див. Projects/l10_union_demo.

- Масиви, об'єднання та структури можна комбінувати практично як завгодно.
- Ініціалізувати об'єднання можна лише тим елементом, тип якого записано першим:

```
union U{
    int n;
    char str[10];
};

union U u1 = {5};
```

- Зауважте, у прикладі вище, код:

```
union U u2 = {"abc"};
```

в чистому C скомпілюється, (в C++ – ні!), але буде поводитися неочікувано. Як саме підкаже попередження компілятора[6]:

```
Warning! W102: Type mismatch (warning)
Note! N2003: source conversion type is 'char *'
Note! N2004: target conversion type is 'int '
```

[6] На прикладі Open Watcom

Анонімні об'єднання

- Якщо створити об'єднання, не задавши явно ні його імені, ні імені поля, відповідні поля будуть доступні безпосередньо.

```
struct D {
    char c;
    int n;
    union {
        float anond;
        int anone;
    };
};
    .................
D d;
d.anond = 3.14;
```

- При тому, об'єднання залишається об'єднанням, всі його поля займають ту саму пам'ять.
  - ◇ Зокрема, у прикладі вище, можна використовувати або anond або anone.
- Такі об'єднання можуть мати лише публічні не статичні члени даних – решта заборонене.
- Об'єднання та класи із об'єднаннями називають збірно Union-like classes.

Об'єднання – підсумок

- Хоча об'єднання іноді потрібні та корисні, в цілому, вони занадто низькорівневі і погано взаємодіють із засобами C++.
- Тому, варто дуже обережно підходити до їх використання.
- C++ має альтернативний, але значно безпечніший, інструмент – std::variant[7], на біду – std::any.
- Можливе використання union – використання тієї ж статично виділеної пам'яті, у різні моменти часу життя програми, для різних цілей. Може бути корисним в embedded, але, скажімо, це зручний спосіб реалізувати small string optimization.
- Використання для перетворення типів (наприклад, int в послідовність байт) підтримується деякими компіляторами і в деяких середовищах є популярним засобом. Якщо у вас ця ситуація – проблем немає. Однак, в цілому, таке використання, згідно стандарту – невизначена поведінка.

[7] Хоча, variant-подібні типи зручно реалізовувати як tagged union – анонімне об'єднання у класі, який містить тег – ідентифікатор активного члену.

Об'єднання – посилання

- "What's with the weird wReserved value at the start of the DECIMAL structure?" – есей про фокуси з об'єднаннями. Зокрема, про "common initial sequence". Також продовження: "A consequence of the weird wReserved value at the start of the DECIMAL structure".
- "C++26: std::is_within_lifetime" – засіб для роботи з union під час компіляції.

Бітові поля

Є можливість виділити під певні величини рівно стільки біт, скільки їм потрібно. Робиться це за допомогою бітових полів — сутностей, що дуже схожі на структури, але задають величини своєрідних полів у бітах:

```
struct BF
{
    unsigned int is_tested : 1;
    unsigned int is_valid : 1;
    unsigned int rating : 3;
};
```

```
BF b1;
int z;
b1.is_tested = 1;
b1.rating = 3;
z = b1.is_tested;
```

- Тип таких полів має бути цілим.
- Варто явно конкретизувати "знаковість" (часто хотілося б, щоб поля були беззнаковими!).
- Ширина поля в бітах задається після двокрапки.

Увага!

- Фізичне розташування бітів бітового поля у машинному слові системно-залежне! Вони можуть йти як з початку, так і з кінця слова. Програми, які покладаються на цей порядок – не переносимі.
- Не на всіх системах бітові поля можуть перетинати границі слова.
- Для вирівнювання наступного поля на границю розміру int, можна створити умовне безіменне поле з розміром нуль:

  unsigned int : 0;

- Також можна створювати безіменні поля для вирівнювання бітів у слові:

  unsigned int : 5;

  Але не слід забувати про системозалежність...

- В C++20 додали можливість безпосередньо ініціалізувати бітові поля при оголошенні:

```
struct BF
{
    unsigned int is_tested : 1 {0};
    unsigned int is_valid : 1 = 1;
    unsigned int rating : 3 = -1;
};
```

- "Bitfield Pitfalls" – приклад труднощів, пов'язаних з інтерпретацією їх як int/unsigned.